



El Cifrado César

DESDE LA ANTIGUA ROMA HASTA LA CRIPTOGRAFÍA
MODERNA

Integrantes:

Aurelio Rendon Viciago - 220584335

Alejandro Rosas Arreola - 222967177

Humberto de Jesús Peña Dueñas - 223992329

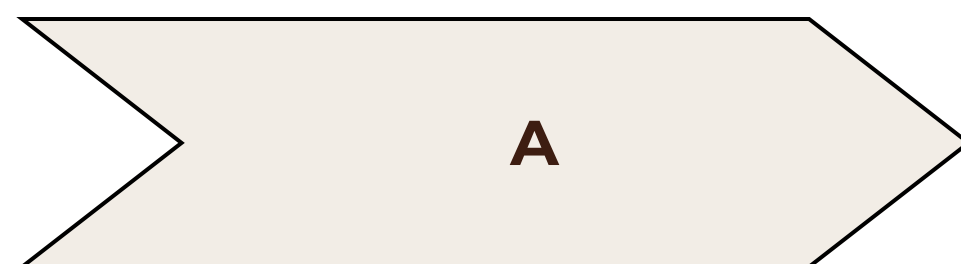


Historia y Fundamentos

El Cifrado César es un método de cifrado por sustitución simple, uno de los más antiguos y conocidos en la historia de la criptografía. Su nombre proviene de Julio César, quien lo utilizaba para proteger sus comunicaciones militares.

Este algoritmo se basa en el desplazamiento de cada letra del mensaje original un número fijo de posiciones en el alfabeto. La clave es ese número de desplazamiento, que solo el emisor y el receptor deben conocer.

¿Cómo funciona el Cifrado César?



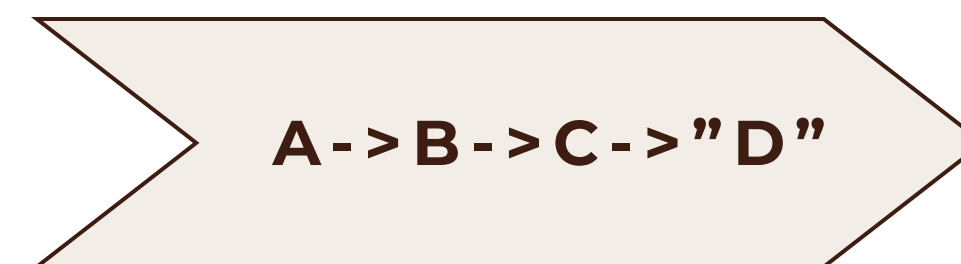
Alfabeto Original

Se toma cada letra del mensaje en claro.



Desplazamiento

Se aplica un desplazamiento constante (la clave) a cada letra.



Alfabeto Cifrado

La letra se sustituye por la que resulta de la posición desplazada.

Por ejemplo, con un desplazamiento de 3, la 'A' se convierte en 'D', la 'B' en 'E', y así sucesivamente. Al llegar al final del alfabeto, se "vuelve" al principio (cifrado modular).

Implementación en Python: Cifrado

Función `cifrar(palabra)`

Esta función toma una palabra y la cifra utilizando un desplazamiento aleatorio entre 1 y 25. Convierte todo a minúsculas y maneja los caracteres especiales como espacios y la letra 'ñ'.

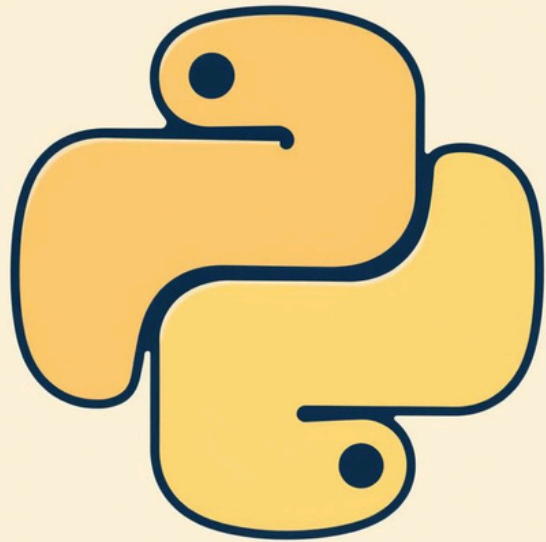
```
def cifrar(palabra):  
    cifrada = []  
    salto = random.randint(1, 25)  
    palabraMinuscula = palabra.lower()  
    for letra in palabraMinuscula:  
        char = int(ord(letra))  
        if (char != 32): # Maneja espacios  
            if (char == 241 or char ==  
209): # Maneja ñ Ñ  
                char = definirChar(110 +  
salto)  
            else:  
                char = definirChar(char +  
salto)  
        cifrada.append(chr(char))  
    return "".join(cifrada)
```

`definirChar(ascii)`

Esta función se asegura de que cualquier valor ASCII que exceda la letra 'z' (122) se ajuste para que vuelva al inicio del rango de letras minúsculas ('a' = 97). Es decir, si el desplazamiento lo lleva más allá de 'z', la función lo reinicia desde 'a', manteniéndolo siempre dentro del rango válido de letras minúsculas.

```
def definirChar(ascii):  
    if (ascii >= 123):  
        return ascii - 122 + 96  
    return ascii
```

Descifrando con Fuerza Bruta



Función `cifradoCesar(cifrada)`

Esta función simula un ataque de fuerza bruta. Itera a través de los 25 posibles desplazamientos (claves) y muestra el resultado de descifrar el mensaje con cada uno, revelando el texto original en alguna de las iteraciones.

```
def cifradoCesar(cifrada):  
    iteraciones = []  
    for i in range(1, 26):  
        for letra in cifrada:  
            char = int(ord(letra))  
            if (char != 32): # Con esto maneja los espacios  
                if (char == 241 or char == 209): # Con esto maneja las ñ Ñ  
                    char = definirChar(110 + i)  
                else:  
                    char = definirChar(char + i)  
            iteraciones.append(chr(char))  
    print(f"Iteracion {i} . _ {"".join(iteraciones)}")  
    iteraciones = []
```

Ejecución y Resultados

Veamos cómo se comporta nuestro algoritmo con un mensaje de prueba.

Mensaje Original:

AnaLiSis de AlGoritMoS

La función `cifrar()` generará un mensaje cifrado con un desplazamiento aleatorio.

```
prueba = cifrar("AnaLiSis de AlGOritMoS")  
cifradoCesar(prueba)
```

Posteriormente, `cifradoCesar()` aplicará los 25 posibles descifrados, mostrando en qué iteración se revela el mensaje legible.

Iteraciones

Cifrada es = rerczjzj uv rcxfizkdfj

Iteracion 1 ._ sfsdakak vw sdygjalegk

Iteracion 2 ._ tgteblbl wx tezhkbmfhl

Iteracion 3 ._ uhufcmcm xy ufailcngim

Iteracion 4 ._ vivgdndn yz vgbjmdohjn

Iteracion 5 ._ wjwheoeo za whcknepiko

Iteracion 6 ._ xkxifpfp ab xidlofqjlp

Iteracion 7 ._ ylyjgqqg bc yjempgrkmq

Iteracion 8 ._ zmzkhrhr cd zkfnqhslnr

Iteracion 9 ._ analisis de algoritmos

Iteracion 10 ._ bobmjtjt ef bmhpsjunpt

Iteracion 11 ._ cpcnkuku fg cniqtkvoqu

Iteracion 12 ._ dqdolvlv gh dojrulwprv

Iteracion 13 ._ erepmwmw hi epksvmxqsw

Iteracion 14 ._ fsfqnxnx ij fqltwnyrtx

Iteracion 15 ._ gtgroyoy jk grmuxozsuy

Iteracion 16 ._ huhspzpz kl hsnvypatvz

Iteracion 17 ._ ivitqaqa lm itowzqbuwa

Iteracion 18 ._ jwjurbrb mn jupxarcvxb

Iteracion 19 ._ kxkvscsc no kvqybsdwyyc

Iteracion 20 ._ lylwtdtd op lwrzctexzd

Iteracion 21 ._ mzmxeue pq mxsadufyae

Iteracion 22 ._ nanyvfvf qr nytbevgzbf

Iteracion 23 ._ obozwwgw rs ozucfwhacg

Iteracion 24 ._ pcpaxhxx st pavdgxibdh

Iteracion 25 ._ qdqbyiyi tu qbwehyjcei

Aplicaciones Reales del Cifrado



Mensajes Rápidos y Secretos

Aunque simple, el Cifrado César puede ser útil para proteger mensajes cortos y poco sensibles en situaciones donde la rapidez es más importante que la seguridad robusta.



Educación y Juegos

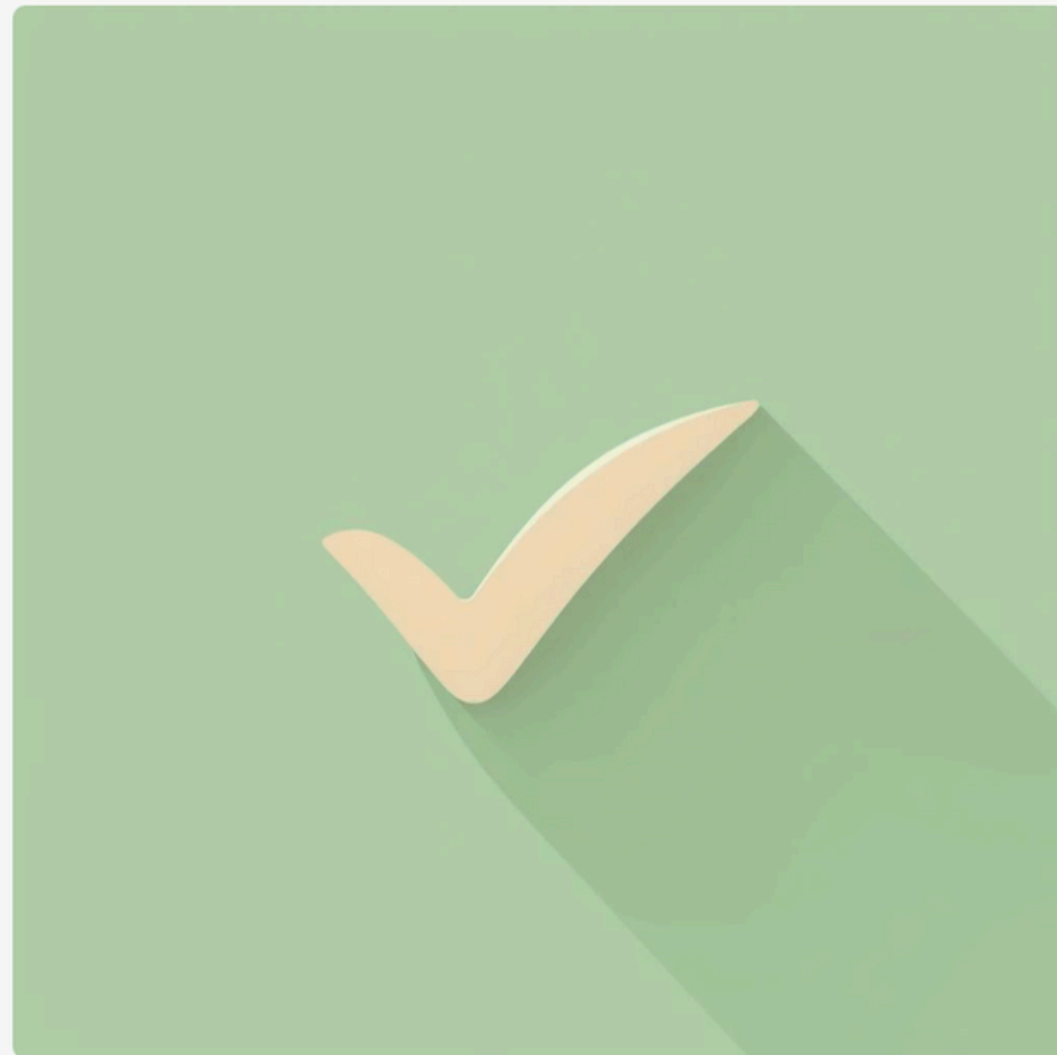
Es una excelente herramienta educativa para introducir los conceptos básicos de la criptografía a estudiantes, o para juegos y acertijos que requieran un cifrado sencillo.



Fortalezas y Debilidades

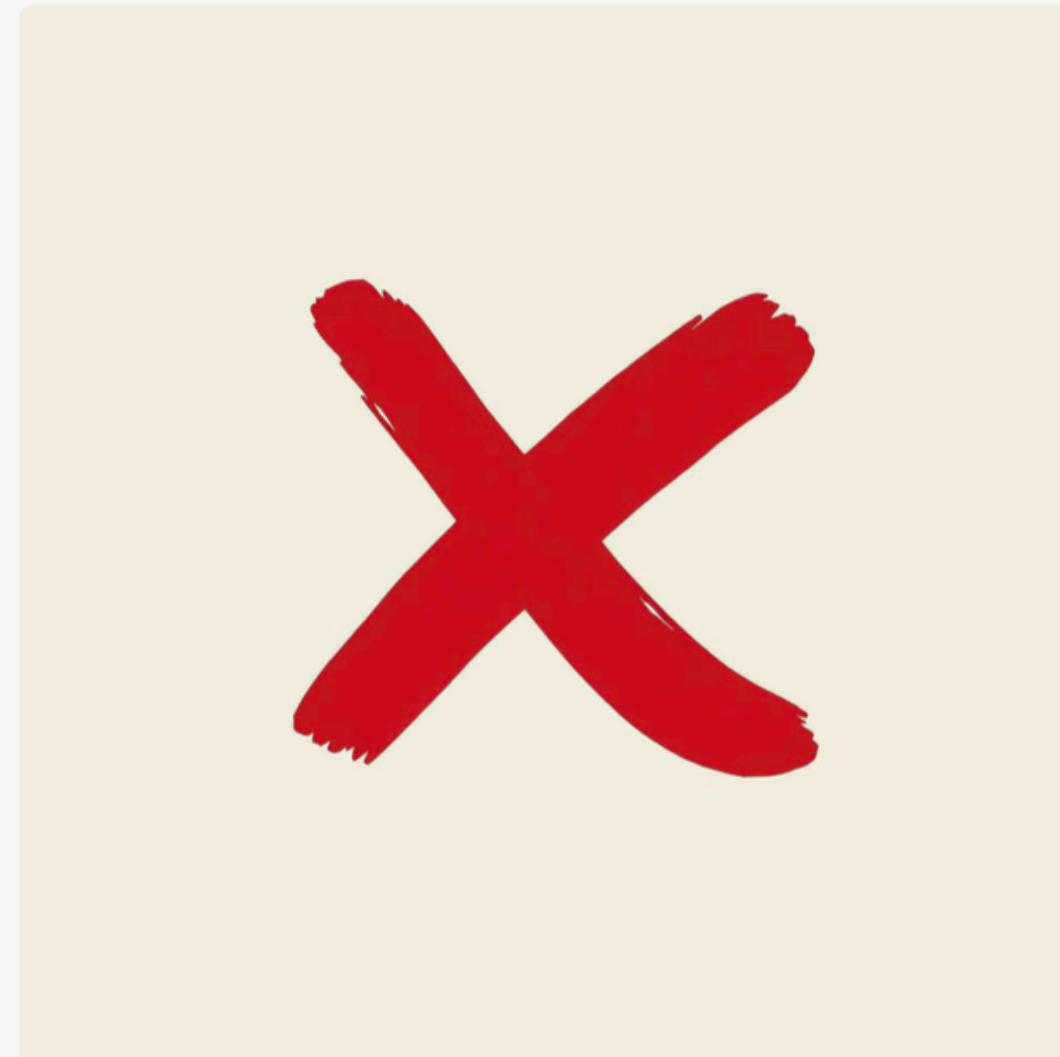
Ventajas

- Fácil de entender e implementar.
- Rápido de cifrar y descifrar manualmente.
- Introduce conceptos criptográficos básicos.



Desventajas

- Muy fácil de romper (pocas claves).
- Vulnerable al análisis de frecuencia.
- No apto para información sensible.



Complejidad Temporal

Aunque el Cifrado César es simple, entender su complejidad nos ayuda a contrastarlo con algoritmos más avanzados.



Tiempo ($O(N)$)

La complejidad temporal es lineal, donde N es la longitud del mensaje. Cada carácter se procesa una vez.



Espacio ($O(N)$)

La complejidad espacial también es lineal, ya que se almacena el mensaje cifrado de la misma longitud que el original.



Comparación

Mucho menos eficiente en seguridad que algoritmos modernos como AES, pero también mucho menos costoso computacionalmente.



Evolución de la Criptografía en la Era Computacional

Evolución de la Criptografía

- Cifrado César: primeros métodos, desplazamiento de letras.
- Sustitución y Transposición: mayor complejidad.
- Criptografía Simétrica: misma clave para cifrar y descifrar (ej. DES, AES).
- Criptografía Asimétrica (RSA): clave pública y privada, base de la seguridad en internet.
- Avances recientes: ECC y criptografía post-cuántica para enfrentar computadoras del futuro.

Conclusión y Reflexiones

Finales

El Cifrado César es un algoritmo didáctico y sencillo, ideal para introducirnos en el mundo de la criptografía. Sin embargo, su vulnerabilidad lo hace inadecuado para la seguridad moderna.

Es recomendable usarlo para fines educativos, juegos o mensajes con muy baja necesidad de seguridad. Para cualquier otro propósito, existen alternativas criptográficas mucho más robustas.

Las posibles mejoras irían en la línea de aumentar el espacio de claves, o combinarlo con otras técnicas, lo que nos llevaría a algoritmos más complejos y seguros.

